

Introduction to Algorithms: 6.006 (算法导论)

Massachusetts Institute of Technology (麻省理工学院) Instructors (授课教师): Erik Demaine (埃里克·德梅因), Jason Ku (顾杰森), and Justin Solomon (贾斯汀·所罗门)

[session] Solution: Quiz 2 (测验 2 解答)

MIT 6.006 Spring 2020, Quiz 2 Solutions (测验 2 解答) · 主题: 图算法综合 (SSSP/APSP/环检测) · 对应 Lecture 9-14 与 CLRS §22 (图的基本算法)、§24 (单源最短路径)、§25 (全对最短路径)

[notice] 测验说明 (Instructions)

- 在被告知开始之前, 不要打开这本测验小册子。阅读本页上的所有说明。
- 测验开始时, 在这本测验小册子的每一页顶部写下你的名字。
- 你有 90 分钟时间, 最多可得 90 分。不要在任何一个问题上花太多时间。先快速浏览所有问题, 然后按能让你取得最大进展的顺序作答。
- 你可以带两张双面 letter-size 的笔记纸。不允许使用计算器、手机或其他可编程或通信设备。
- 在提供的空白处写下你的解答。页面将被扫描并按题分离评分。如果你需要更多空间, 请写 "Continued on S1" (或 S2、S3), 并在试卷末尾标注的草稿页上继续你的解答。
- 不要浪费时间与纸张去重新推导我们在 lecture、recitation 或 problem sets 中已经学过的结论。直接引用它们即可。
- 写算法时, 用清晰的英文描述即可, 不需要伪代码。务必论证你的算法是正确的, 并分析算法的渐近运行时间。即使你的算法不满足要求的时间上界, 正确但低效的解答也可能获得部分分。
- 密切关注每个问题的说明。根据问题的不同, 不完整的答案也可能获得部分分。

Problem (题号)	1: Information (信息)	2: Exact Edges (精 确边数)	3: Color Cost (颜 色代价)	4: Orkscapade (兽人大冒险)	5: Count Cycles (环 计数)	6: Bellham's Fjord (贝尔汉姆 的峡湾)	Total (总 分)
Parts (小 题数)	2	1	1	1	1	1	
Points (分 值)	2	16	18	18	18	18	90

Name (姓名) : School Email (学校邮箱) :

[problem] Problem 1. Information (信息) [2 points]

(a) [1 point] 在封面页写上你的姓名和邮箱地址。

[solution] Solution (解答) : 好的!

(b) [1 point] 在每页顶部写上你的名字。

[solution] Solution (解答) : 好的!

[problem] Problem 2. Exact Edges (精确边数) [16 points]

给定一个带正负 edge weights (边权) 的 weighted, directed graph (带权有向图) $G = (V, E, w)$, 以及一个特定的 vertex $v \in V$, 描述一个 $O(k|E|)$ 时间的 algorithm, 返回包含 vertex v 且恰好有 k 条 edges 的任何 cycle (环) 的最小权重, 若不存在这样的 cycle 则返回不存在。回忆: 一个 cycle 可以重复 vertices/edges。

[solution] Solution (解答) : 假设 G 中所有 vertices 都从 v 可达, 从而 $|V| = O(|E|)$; 否则, 运行 BFS 或 DFS 求解从 v 出发的 single source reachability (单源可达性), 把 G 替换为从 v 可达的子图, 耗时 $O(|E|)$ 。构造一个新图 $G' = (V', E')$, 其中:

- 对每个 vertex $v \in V$ 有 $k + 1$ 个 vertices: 具体地, 对所有 $i \in \{0, \dots, k\}$ 有 v_i ; 以及
- 对每条 edge $(u, v) \in E$ 有 k 条 edges: 具体地, 对所有 $i \in \{1, \dots, k\}$ 有 edges (u_{i-1}, v_i) 。

图 G' 有 $(k + 1)|V| = O(k|E|)$ 个 vertices, 在 $k + 1$ 层中有 $k|E|$ 条 edges, 并且具有性质: 从 v_0 到 v_k 的 paths 与 G 中经过 v 、权重相同、恰好包含 k 条 edges 的 cycles 一一对应——因为每增加一层恰好穿越一条 edge。

所以从 v_0 求解 SSSP, 返回到 v_k 的最小权重 path。由于 G' 中的 edges 总是使下标递增, G' 是一个 DAG, 因此可以用 DAG relaxation (DAG 松弛) 在关于 G' 大小的线性时间内求解 SSSP。所以连同最初的剪枝, 该算法总共耗时 $O(k|E|)$ 。

Common Mistakes (常见错误) :

- 用 BFS 检测有向图中的 cycles (应该用 DFS)
 - 试图枚举长度为 k 的所有 paths 或 cycles (可能是指数级)
 - 使用 Bellman-Ford 而不移除零权重 edges (可能使用少于 k 条 edges)
 - 用 $k - 1$ 条 edges 而不是 k 条 edges 找 cycles
 - 试图找 negative-weight cycles (找错了问题)
 - 不运行 reachability (可达性) 算法把图剪枝到从 v 可达的子图
-

[note] Note (译者注) : 这道题的核心是"用分层图把'恰好 k 条边'变成普通最短路": vertex u_i 表示"走了恰好 i 条边后到达 u ", 边 (u_{i-1}, v_i) 让"走一条边"恰好把层数加一。于是"含 v 且恰好 k 条边的 cycle"变成" G' 中从 v_0 到 v_k 的 path"——注意这里的 cycle 允许重复 vertices/edges, 而分层图天然允许这种重复 (每层都是同一个原始 vertex 的副本, 可以反复经过), 这正是答案说"一一对应"的原因。这个技巧你在 PS6-2 (Short Circuits) 刚见过: 那里是"至多 k 条边"所以逐层增量构建, 这里是"恰好 k 条边"所以建满 $k + 1$ 层。另一个关键点是为什么 G' 是 DAG: 所有边都从第 $i - 1$ 层指向第 i 层, 层数严格递增, 不可能有环——因此可以用 $O(|V'| + |E'|) = O(k|E|)$ 的 DAG relaxation 而不是 $O(k|V'||E'|)$ 的 Bellman-Ford。Common Mistakes 里"用 BFS 检测有向图 cycle"值得注意: BFS 只适合检测无向图中的环, 有向图必须用 DFS 的三色标记或拓扑排序。

[problem] Problem 3. Color Cost (颜色代价) [18 points]

图的 3-color labeling (三色标记) 把每条 edge 映射为红色、绿色或蓝色。一条三色标记路径的 color cost (颜色代价) 是其 path weight (路径权重) 加上路径每次改变颜色时的正整数 w_c 。例如, 在下图中 (有四个 vertices $\{a, b, c, d\}$: 一条权重 w_1 的 blue edge (蓝边) (a, b) , 一条权重 w_2 的 red edge (红边) (b, c) , 以及一条权重 w_3 的另一条 blue edge (c, d)), 路径 (a, b, c, d) 的 color cost 是 $w_1 + w_2 + w_3 + 2w_c$, 因为该路径改变了两次颜色。

```
a —w1— b —w2— c —w3— d
  (blue)   (red)   (blue)
```

(图中 w_1 、 w_2 、 w_3 为对应 edges 的权重, 均为 undirected edges (无向边); 示例路径 (a, b, c, d) 的颜色依次为 blue $\rightarrow$ red $\rightarrow$ blue, 改变颜色两次, 故 color cost 为 $w_1 + w_2 + w_3 + 2w_c$)

给定一个连通、带权、undirected graph (无向图) $G = (V, E, w)$ 的 3-color labeling $c : E \rightarrow \{\text{red, green, blue}\}$, 其中只含正 edge weights, 以及两个 vertices $s, t \in V$, 描述一个高效的 algorithm, 返回一条从 s 到 t 的、color cost 最小的 path。(所谓"高效": 越快且正确的算法得分越高。)

[solution] Solution (解答) : 构造一个新图 $G' = (V', E')$, 其中:

- 对每个 vertex $v \in V$ 有 3 个 vertices: 具体地, v_i ($i \in \{\text{red}, \text{green}, \text{blue}\}$) 对应"经由一条颜色为 i 的 edge 到达 vertex v ";
- (vertex-edges (顶点边)) 对每个 vertex $v \in V$ 有 3 条 undirected edges: 具体地, $\{v_{\text{red}}, v_{\text{blue}}\}$ 、 $\{v_{\text{green}}, v_{\text{red}}\}$ 与 $\{v_{\text{blue}}, v_{\text{green}}\}$, 权重均为 w_c ; 以及
- (edge-edges (边边)) 对每条 undirected edge $\{u, v\} \in E$ 有 1 条 undirected edge: 具体地, 颜色为 $c(u, v)$ 、权重为 w 的 undirected edge $\{u_{c(u,v)}, v_{c(u,v)}\}$ 。

图 G' 有 $3|V|$ 个 vertices 和 $3|V| + |E|$ 条 edges, 并具有性质: G' 中从任意 vertex s_i 到任意 vertex t_j ($i, j \in \{\text{red}, \text{green}, \text{blue}\}$) 的任何 path 的最小权重, 等于 G 中从 s 到 t 的任何三色标记 path 的最小 color cost——因为在 vertex 处改变颜色需要穿越一条权重为 w_c 的 edge。所以分别从每个 s_i 运行三次 SSSP, 找出到任意 t_j 的任何 path 的最小权重, 然后如 lecture 所示构造 parent pointers (父指针) 返回一条最小 path。由于该图只有正 edge weights, 可以用 Dijkstra 在 $O(|V| + |E| + |V| \log |V|) = O(|E| + |V| \log |V|)$ 时间内求解 SSSP。

注意: 你可以用一个 supernode (超级节点) 避免运行三次 Dijkstra, 但这只是把工作量减少常数倍。另外, 也可以不给每个 vertex 添加 vertex-edges, 而是给每条相连的 edge 添加三条适当加权的 edges, 但这些 edges 需要指向标有与对应 edge 相同颜色的 vertex。

Common Mistakes (常见错误) :

- 错误地试图修改 Dijkstra 来跟踪状态
 - 未能识别 (或识别错误) 运行 SSSP 的源点
 - 在复制图中错误地加权或定向 edges
-

[note] Note (译者注)：这是"状态展开"技巧的又一实例，但这次的"状态"是到达一个 vertex 时最后一条边的颜色。为什么要分 3 个副本？因为 color cost 的惩罚发生在"换颜色"的瞬间——如果不知道到达 v 时的颜色，就无法判断下一条边是否触发 w_c 。把状态 (vertex, 颜色) 变成 vertex 后，换颜色的代价 w_c 就变成了图里的"内部边" (vertex-edges 把同色副本连起来)，而沿原图走一条边则是在同色副本之间移动 (edge-edges)。你可以把 G' 想成三个平行的"颜色世界"， w_c 边是连接相邻世界的通道。为什么答案是 Dijkstra 而不是 Bellman-Ford？所有权重 (w 和 w_c) 都是正的。Common Mistakes 里"试图修改 Dijkstra 跟踪状态"正是初学者最容易走的弯路——标准 Dijkstra 不能处理"边权依赖历史"的问题，正确做法永远是**把历史状态展开进图结构**。顺带一提：运行三次 Dijkstra 可以用 supernode 合并为一次，但收益只是常数因子，这正是官方解答说"没必要"的原因——这也符合 6.006 对"高效"的定义（渐近意义下）。

[problem] Problem 4. Orkscapade (兽人大冒险) [18 points]

游侠 Raargorn (拉尔戈恩) 需要把一条消息从她的家乡 Tina's Mirth (蒂娜欢愉镇) 送到 Riverdell (里弗德尔镇)，但 Midgard (米德加德) 的城镇已经被一支 k 个 Orks (兽人) 的军队占领。Raargorn 有一张 Midgard 的 n 个城镇和 $3n$ 条道路的地图，每条道路双向连接一对城镇。侦察兵已确定驻扎在每个城镇 i 的 Orks 数量 $r_i \geq 1$ (每个城镇至少驻扎一个 Ork)。描述一个 $O(k)$ 时间的 algorithm，找到一条从 Tina's Mirth 到 Riverdell 的 path，使 Raargorn 沿途在城镇中遇到的 Orks 总数最少。较慢但正确的算法 (例如 $O(k \log k)$ 或 $O(nk)$) 将获得部分分。

[solution] Solution (解答) : 构造图 $G = (V, E)$, 其中:

- 对每个城镇 v 有一条由 r_v 个 vertices $(v_1, \dots, v_{r_v})$ 组成的 chain (链), 由 $r_v - 1$ 条 edges 连接, 即对所有 $i \in \{1, \dots, r_v - 1\}$ 有无权 directed edge (有向边) (v_i, v_{i+1}) ; 以及
- 对城镇 u 和 v 之间的每条 road, 有两条无权 directed edges (u_{r_u}, v_1) 和 (v_{r_v}, u_1) 。

图 G 有 $\sum_v r_v = k$ 个 vertices 和 $2(3n) + \sum_v (r_v - 1) = 5n + k$ 条 edges。由于每个城镇至少有一个 Ork, $k \geq n$, 所以 G 的大小是 $O(k)$ 。令 s 和 t 分别对应 Tina's Mirth 和 Riverdell 这两个城镇。图 G 具有性质: 任何从 s_1 到 t_{r_t} 的 path 都对应一条从 Tina's Mirth 到 Riverdell 的 path, 且穿越的 edges 数 (加 1) 等于沿途城镇中遇到的 Orks 数——因为对连接城镇 u 和 v 的任何 road, 从 u_1 出发到 v_1 需要穿越 r_u 条 edges (u 镇链的 $r_u - 1$ 条边加 1 条 road 边; 原文此处写作 r_v , 应为 r_u , 见译者注)。所以用 BFS 从 s_1 求解到 t_{r_t} 的 unweighted SSSP (无权单源最短路径), 耗时 $O(k)$, 并通过跟随 parent pointers 返回找到的最短 path 上途经的城镇序列。

Common Mistakes (常见错误) :

- 不用 vertex weight 定向 edges, 或以其他方式没有清楚地定义图
 - 把所有 edges 的权重展开, 导致 $O(nk)$ 规模的展开
 - (例如, 一个有 $\Theta(k)$ 个 Orks 的城镇可能连接 $\Theta(n)$ 条 roads)
 - 使用未经修改的 Dijkstra 来达到 $O(k \log k)$ 时间
 - 错误地展开 vertex weights (路径长度为 r_i 而不是 $r_i - 1$)
 - 在 BFS 或 DFS tree 中找最短路径 (其中可能不包含最短路径)
-

[note] Note (译者注) : 这道题展示了一个精巧的"把点权展开成链"的技巧: 城镇 v 有 r_v 个 Orks, 于是把它展开成一条 r_v 个节点、 $r_v - 1$ 条边的链——"路过这个城镇"在展开图中等价于"沿着链走 $r_v - 1$ 条边", 而"从城镇 u 到城镇 v "这条 road 对应从 u 链尾跳到 v 链头的边 (u_{r_u}, v_1) , 之后还要再走 v 的链。这样"遇到的总 Orks 数"就精确等于"展开图中的路径长度", 把 vertex weights (点权) 转化成了 edge count (边数)。为什么点权展开要用 $r_v - 1$ 条边而不是 r_v 条? 因为链有 r_v 个节点, 节点之间的边数是 $r_v - 1$ ——Common Mistakes 专门列了这条, 是最容易犯的 off-by-one (差一错误)。另一个细节: $k \geq n$ (每个城镇至少 1 个 Ork) 保证了 $5n + k = O(k)$, 否则 n 可能远大于 k , $O(k)$ 上界就不成立——这是题目"至少一个 Ork"这一假设的作用。为什么 BFS 够用? 展开图中所有边权重相同 (都无权), BFS 天然给出最短路径; 错误项"在 BFS tree 中找最短路径"提醒你: BFS tree 的树边不一定构成最短路径, 必须用 parent pointers 从目标反向重构。**最后校正官方解答的两处不严谨:** 一是 "going from u_1 to v_1 requires traversing r_v edges" 中的 r_v 应为 r_u ——从 u_1 到 v_1 走的是 u 镇链的 $r_u - 1$ 条边加 1 条 road 边, 共 r_u 条; 二是"穿越边数 = 遇到的 Orks 数"的精确说法应为"穿越边数 + 1 = Orks 数"——从 s_1 到 t_{r_t} 的路径共有 $\sum_v r_v - 1$ 条边 ($m + 1$ 个城镇的链共 $\sum r_v - (m + 1)$ 条边, 加 m 条 road 边), 而遇到的 Orks 为 $\sum_v r_v$ 个。由于对每条路径这个差都是常数 1, 最小化边数仍等价于最小化 Orks 数, BFS 的正确性不受影响。

[problem] Problem 5. Count Cycles (环计数) [18 points]

一个 cycle-sparse graph (环稀疏图) 是任何 weighted directed simple graph (带权有向简单图) $G = (V, E, w)$, 满足: G 中每个 vertex $v \in V$ 至多从一个 simple (简单) negative-weight cycle (负权环) 可达。给定一个 cycle-sparse graph, 描述一个 $O(|V|^3)$ 时间的 algorithm, 返回 G 中 negative-weight cycles 的数量。

[solution] Solution (解答) : 构造新图 G' : 向 G 添加一个 supernode (超级节点) x , 对每个 $v \in V$ 加一条零权重 directed edge (有向边) (x, v) 。然后用 Bellman-Ford 从 x 在 G' 中求解 SSSP, 给每个 vertex $v \in V$ 标上其最短路径距离 $\delta(x, v)$ 。对每个 $v \in V$, $\delta(x, v) = -\infty$ 当且仅当 v 从 G 中的某个 negative-weight cycle 可达 (因为添加 x 不会增加或移除任何 cycles)。进一步, 对任何 directed edge (u, v) , 如果 $\delta(x, u) = \delta(x, v) = -\infty$, 那么 u 和 v 各自都从同一个 simple negative-weight cycle 可达 (因为 v 从 u 可达, 且每个 vertex 至多从一个 simple negative-weight cycle 可达)。

所以, 构造一个新图 G'' : 只包含 G' 中满足 $\delta(x, v) = -\infty$ 的 vertices $v \in V$, 若 u 和 v 在 G 中共享一条 directed edge, 则在 G'' 中 u 和 v 之间放一条 undirected edge。图 G'' 具有性质: G'' 中 connected components (连通分量) 的数量等于 G 中 negative-weight cycles 的数量, 所以用 Full-BFS 或 Full-DFS 统计并返回 G'' 中 connected components 的数量。该算法构造 G' 耗时 $O(|V| + |E|)$, 运行 Bellman-Ford 耗时 $O(|V||E|)$, 构造 G'' 耗时 $O(|V| + |E|)$, 然后统计 G'' 中的 connected components 耗时 $O(|V| + |E|)$, 总运行时间 $O(|V||E|) = O(|V|^3)$ 。

Common Mistakes (常见错误) :

- 描述算法时普遍缺乏精确性
- 试图枚举所有 paths 或 cycles (可能是指数级)
- 反复运行 Bellman-Ford, 通常产生 $|V| \cdot O(|V||E|) = O(|V|^4)$ 时间
- 声称 $|E| = O(|V|)$
- 混淆 connected components (连通分量) 与 strongly connected components (强连通分量)

Footnote (脚注) : 回忆: 如果某个 cycle 至多访问每个 vertex 一次, 则它是 simple 的。

[note] Note (译者注) : 这题最反直觉的一步是把"数负权环"转化为"数连通分量"。核心杠杆是 cycle-sparse 性质：每个 vertex 至多从一个 simple 负权环可达。由此，凡是 $\delta(x, v) = -\infty$ 的 vertex 都可以**唯一**地归属到某一个负权环——如果 $u \rightarrow v$ 有边且两者都是 $-\infty$ ，那么 v 从 u 可达，而 u 、 v 各自只能对应同一个环（否则 v 会从两个环可达，违反 cycle-sparse）。因此"从同一环可达"是一个等价关系（你在 6.042J 学过的 equivalence relation（等价关系））， G'' 中每条这样的边把属于同一环的 vertices 连成连通块，块数 = 环数。为什么不能直接数 $-\infty$ 的 vertex？因为一个环可能让多个 vertices 变 $-\infty$ （环上的每个 vertex 都从该环可达），直接数会重复计数——必须先"合并同类项"。Common Mistakes 里的"混淆 connected components 与 strongly connected components"很关键： G'' 是无向图，用 connected components；若误用 SCC 会把"环上的节点集合"与"环的强连通区域"混淆。时间上界 $O(|V|^3)$ 来自 Bellman-Ford 的 $O(|V||E|)$ 加上最坏情形 $|E| = O(|V|^2)$ 。

[problem] Problem 6. Bellham's Fjord (贝尔汉姆的峡湾) [18 points]

Gralexandra Bellham（格拉莱珊德拉·贝尔汉姆）想开她的电动汽车在挪威从奥斯陆的 s 位置前往 t 位置的一处风景优美的 Fjord（峡湾）。她有一张挪威 n 个位置的地图，以及 $O(n)$ 条直接连接位置对的单向道路。

- 每个位置 x 标有其（正或负）整数 height（海拔） $h(x)$ （海平面以上）。
- 她的汽车有 regenerative braking（再生制动），下坡时可以发电。每条从位置 x 到 y 的道路标有整数 energy（能量） $J(x, y)$ ，电动汽车沿路行驶时消耗（正）或产生（负）这么多能量。
- 根据物理定律， $J(x, y)$ 总是严格大于位置 x 与 y 之间的 potential energy（势能）差，即 $J(x, y) > m \cdot g \cdot (h(y) - h(x))$ ，其中 m 和 g 分别是汽车的质量与重力加速度。

她的汽车电池有非常大的 energy capacity（能量容量） $b > 2nk$ ，其中 k 是任意 road 的最大 $|J(x, y)|$ 。假设她以半容量 $\lfloor b/2 \rfloor$ 从 s 出发，描述一个 $O(n \log n)$ 时间的 algorithm，确定 Bellham 到达 t 时电池中能拥有的最大能量。较慢但正确的算法（例如 $O(n^2)$ ）将获得部分分。

[solution] Solution (解答) : 构造图 G : 挪威的 n 个位置各一个 vertex, $O(n)$ 条道路各一条 directed edge (有向边): 具体地, 对每条从位置 u 到 v 的道路, 添加权重为 $J(u, v)$ 的 directed edge (u, v) 。那么 $\lfloor b/2 \rfloor$ 减去 G 中从 s 到 t 的最小权重 path 的权重, 就对应 Bellham 到达 t 时能拥有的最大能量; 至少, 如果她沿途既不超过也不耗尽油箱, 就会是这样。

首先我们证明 G 中从 s 到 t 的每条最小权重 path 都是 simple (简单) 的。只需证明 G 中每个 directed cycle 都有正权重。考虑 cycle $(c_0, \dots, c_{k-1}, c_k = c_0)$ 。该 cycle 的权重为 $\sum_{i=1}^k J(c_{i-1}, c_i) > \sum_{i=1}^k mg(h(c_i) - h(c_{i-1})) = 0$, 如所愿。

G 中任何 simple path 至多穿越 $n - 1$ 条 edges, 所以其权重的绝对值至多为 $(n - 1)k < b/2$ 。因此 $\lfloor b/2 \rfloor$ 减去 G 中任何 simple path 的权重总是 > 0 且 $< b$ (所以 Bellham 沿从 s 到 t 的 simple path 行驶时既不会耗尽也不会超过电池容量)。

最后, 通过求解 SSSP 找出从 s 到 t 的最小权重 path 的权重。不幸的是, 用 Bellman-Ford 需要 $O(n^2)$ 时间, 太慢了。然而, 我们可以利用提供的 vertex potentials (顶点势), 像 Johnson's algorithm (约翰逊算法) 那样重加权 G 中的 edges 使其为正, 同时保持最短路径不变。具体地, 创建新图 G' , 与 G 相同, 只是把每条 edge (u, v) 的权重改为 $J(u, v) - mg(h(v) - h(u)) > 0$ 。这个变换保持最短路径不变, 因为例如从 a 到 b 的每条 path 的权重都改变相同的量, 即 $mg(h(b) - h(a))$ 。所以从 s 运行 Dijkstra, 找出 G' 中到 t 的任何 path 的最小权重 D , 返回 $\lfloor b/2 \rfloor - (D - mg(h(b) - h(a)))$ 。

构造 G 耗时 $O(n)$, 重加权到 G' 也耗时 $O(n)$, 然后从 s 在 G' 中运行 Dijkstra 耗时 $O(n \log n)$, 总时间 $O(n \log n)$ 。

Common Mistakes (常见错误) : (Problem 6 的常见错误, 续于 S1)

- 直接使用 Bellman-Ford, 得到 $O(n^2)$ 时间的算法 (可得一半分数)
 - 运行 Bellman-Ford 来求新权重, 仍然导致 $O(n^2)$ 时间
 - 对 weights 用 $-J$ 而不是 J
 - b 倍图复制 (低效, b 可能远大于 n)
-

[note] Note (译者注) : 这道题是 Quiz 2 的压轴题，把三个看似无关的知识点串成一条线。**第一**，物理定律 $J(x, y) > mg(h(y) - h(x))$ 意味着“沿任何环走一圈，总能量严格为正”（势能绕一圈回到原点，和为零，而每段消耗都严格大于对应的势能差）——所以图中没有负权环，Bellman-Ford 不会失效，且任何最短路径都是 simple path。**第二**，为什么答案关心“不超过电池容量”？因为 $\lfloor b/2 \rfloor - (n-1)k > 0$ 且 $< b$ ——simple path 的权重绝对值被 k 和 n 夹住，而 $b > 2nk$ 保证了半箱油足够任何简单路径用，于是“电池容量”约束自动满足，问题退化为普通最短路。**第三**，直接 Bellman-Ford 是 $O(n^2)$ ，太慢，所以用 Johnson 的**重加权 (reweighting)** 技巧： $\hat{w}(u, v) = J(u, v) - mg(h(v) - h(u)) > 0$ 使所有边权变正，而每条 $s \rightarrow t$ 路径的权重整体平移 $mg(h(t) - h(s))$ （望远镜和），最短路径不变。Demaine 在 Lecture 14 里这样讲重加权的动机：“In other words, just reweight the edges here so that... after we reweight, we'll go to some graph g' prime with the same combinatorial structure, just different edge weights.”（翻译：换句话说，只要在这里重加权边，使得.....重加权之后我们会得到一个组合结构相同、只是边权不同的图 g' 。）——这正是你在 PS6-6 用代码实现过的 Johnson's Algorithm 的思想，quiz 里以物理题的面目复现。注意最后一步返回 $\lfloor b/2 \rfloor - (D - mg(h(b) - h(a)))$ ： D 是在 G' 中算出的，必须先减回 $mg(h(t) - h(s))$ 还原成 G 中的最小路径权重，再做减法。

[notice] 草稿页 (Scratch Paper)

以下页面是考试草稿纸 (SCRATCH PAPER)，不参与评分。若你在题目的答题空间写不下，可在草稿纸上继续书写，但务必在题目页写上“Continued on S1”（或 S2、S3）。

SCRATCH PAPER 1. DO NOT REMOVE FROM THE EXAM. (草稿纸 1，请勿从试卷上撕下。) (Problem 6 的 Common Mistakes 见上。)

SCRATCH PAPER 2. DO NOT REMOVE FROM THE EXAM. (草稿纸 2，请勿从试卷上撕下。)

SCRATCH PAPER 3. DO NOT REMOVE FROM THE EXAM. (草稿纸 3，请勿从试卷上撕下。)